

Tata Play Fiber Self-Onboarding Middleware Backend

Low-Level Design (LLD) Document

This document provides the low-level design details, API specs, database schemas, and technical implementation components for the Tata Play Fiber Onboarding Middleware Backend.

1. Database Schema Design (PostgreSQL DDL Reference)

```
-- Central Master Table
CREATE TABLE onboarding_master (
    prospect_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rmn VARCHAR(15) NOT NULL,
    email VARCHAR(255) NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    current_step VARCHAR(50) NOT NULL, -- Enum values (e.g. STEP_1_FEASIBILITY)
    status VARCHAR(30) NOT NULL,      -- IN_PROGRESS, COMPLETED, NRFS, FAILED
    customer_id VARCHAR(50),          -- Synced from HOBS
    account_id VARCHAR(50),           -- Synced from HOBS
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Transition Audit Log
CREATE TABLE onboarding_transition_log (
    id BIGSERIAL PRIMARY KEY,
    prospect_id UUID REFERENCES onboarding_master(prospect_id),
    from_step VARCHAR(50) NOT NULL,
    to_step VARCHAR(50) NOT NULL,
    transition_time TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    payload_snapshot JSONB
);

-- Step 1 Table: Lead Details
CREATE TABLE lead_details (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    rmn VARCHAR(15) NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL,
    full_address TEXT NOT NULL,
    latitude NUMERIC(9, 6),
    longitude NUMERIC(9, 6),
    is_feasible BOOLEAN DEFAULT FALSE,
    build_type VARCHAR(100),
    address_status VARCHAR(20) NOT NULL, -- FEASIBLE, NRFS
    status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Step 2 & 3 Table: OTP Verification
CREATE TABLE otp_verification (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    rmn VARCHAR(15) NOT NULL,
```

```

email VARCHAR(255) NOT NULL,
otp_hash VARCHAR(255) NOT NULL,
expiry_time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
attempts_count INT DEFAULT 0,
is_verified BOOLEAN DEFAULT FALSE,
status VARCHAR(30) NOT NULL, -- OTP_SENT, OTP_VERIFIED, EXPIRED, FAILED
verified_at TIMESTAMP WITHOUT TIME ZONE
);

-- Step 4 Table: Document Collection
CREATE TABLE document_collection (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    poi_type VARCHAR(50) NOT NULL,
    poi_document_number VARCHAR(100) NOT NULL,
    poi_file_path VARCHAR(500) NOT NULL,
    poa_type VARCHAR(50) NOT NULL,
    poa_document_number VARCHAR(100) NOT NULL,
    poa_file_path VARCHAR(500) NOT NULL,
    live_photo_file_path VARCHAR(500) NOT NULL,
    declaration_form_file_path VARCHAR(500) NOT NULL,
    trust_validation_status VARCHAR(50) NOT NULL, -- SUCCESS, FAILED
    trust_validation_remarks TEXT,
    is_migrated BOOLEAN DEFAULT FALSE,
    status VARCHAR(30) NOT NULL, -- PENDING, UPLOADED, VERIFIED, FAILED, MIGRATED
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Step 5 Table: Profile Building
CREATE TABLE profile_building (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    gst_number VARCHAR(15),
    company_name VARCHAR(150),
    gst_validation_status VARCHAR(30), -- SUCCESS, FAILED, N/A
    additional_profile_data JSONB,
    status VARCHAR(30) NOT NULL, -- PENDING, GST_VERIFIED, GST_FAILED, SUBMITTED
    updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Step 7 & 8 Table: Plan Selection
CREATE TABLE plan_selection (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    package_id VARCHAR(50) NOT NULL,
    package_name VARCHAR(150) NOT NULL,
    base_price NUMERIC(10, 2) NOT NULL,
    applied_offers JSONB,
    total_discount NUMERIC(10, 2) DEFAULT 0.00,
    total_tax NUMERIC(10, 2) DEFAULT 0.00,
    final_payable_amount NUMERIC(10, 2) NOT NULL,
    status VARCHAR(30) NOT NULL, -- PENDING, SELECTED, OVERRIDDEN
    updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Step 9 Table: Payment Details
CREATE TABLE payment_details (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    customer_id VARCHAR(50),
    account_id VARCHAR(50),
    payment_link VARCHAR(1000),
    link_expiry TIMESTAMP WITHOUT TIME ZONE,
    payment_status VARCHAR(30) NOT NULL, -- PENDING, PAID, EXPIRED
    hobs_sync_status VARCHAR(30) NOT NULL, -- SUCCESS, PENDING, FAILED

```

```

        six_d_sync_status VARCHAR(30) NOT NULL, -- SUCCESS, PENDING, FAILED
        generation_attempts INT DEFAULT 0,
        status VARCHAR(30) NOT NULL, -- PENDING, INITIATED, SUCCESS, EXPIRED, FAILED
        created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

-- Step 11 Table: Consent Details
CREATE TABLE consent_details (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    whatsapp_message_sid VARCHAR(100),
    consent_status VARCHAR(30) NOT NULL, -- PENDING, APPROVED, REJECTED
    status VARCHAR(30) NOT NULL,
    consent_received_at TIMESTAMP WITHOUT TIME ZONE
);

-- Step 12 Table: CAF Details
CREATE TABLE caf_details (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    caf_pdf_url VARCHAR(500) NOT NULL,
    status VARCHAR(30) NOT NULL, -- PENDING, GENERATED, FAILED
    generated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Step 13 Table: EKYC Details
CREATE TABLE ekyc_details (
    prospect_id UUID PRIMARY KEY REFERENCES onboarding_master(prospect_id),
    hobs_ticket_id VARCHAR(50) NOT NULL,
    ekyc_status VARCHAR(50) NOT NULL,
    status VARCHAR(30) NOT NULL, -- TICKET_CREATED, COMPLETED, FAILED
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

---
```

2. API Endpoints & Payload Contracts

All API requests and responses utilize JSON payloads, structured with standard envelopes.

Step 1: Feasibility Check (`POST /api/v1/onboarding/feasibility`)

- ****Request****:

```

{
  "rmn":
  "cus":
  "ema":
  "add":
  "b":
  "s":
  "c":
  "p":
  "l":
  "l":
}
```

- ****Response (Feasible - HTTP 201 Created)****:

```

{
```

```

"sta
"dat
"pr
"fe
"br
"ne
}
}

```

- **Response (Not Feasible - HTTP 422 Unprocessable Content)**:

```

{
  "sta
  "err
  "co
  "m
  for futur
  },
  "dat
  "pr
  "fe
  }
}

```

Step 2: Send OTP (`POST /api/v1/onboarding/otp/send`)

- **Request**:

```

{
  "pro
}

```

- **Response (HTTP 200 OK)**:

```

{
  "sta
  "dat
  "me
  }
}

```

Step 3: Validate OTP & State Recovery (`POST /api/v1/onboarding/otp/validate`)

- **Request**:

```

{
  "pro
  "otp
}

```

- **Response (New Onboarding - HTTP 200 OK)**:

```

{
  "sta
  "dat
  "on
  "re
  "st
  }
}

```

- **Response (Resumed Onboarding - HTTP 200 OK)**:

```

{

```

```

"sta
"dat
"on
"re
"st
}
}
}

```

Step 4: Document Collection (`POST /api/v1/onboarding/{prospectId}/documents`)

- **Content-Type**: `multipart/form-data`
- **Form Parameters**:
 - `poiType`: "AADHAAR"
 - `poiDocumentNumber`: "1234-5678-9012"
 - `poiFile`: (Binary)
 - `poaType`: "AADHAAR"
 - `poaDocumentNumber`: "1234-5678-9012"
 - `poaFile`: (Binary)
 - `livePhoto`: (Binary)
- **Response (HTTP 200 OK)**:

```

{
  "sta
  "dat
  "po
  "po
  "f
  "https://
  }
}

```

Step 5: Profile Building & GST Validation (`POST /api/v1/onboarding/{prospectId}/profile`)

- **Request**:

```

{
  "gst
  "com
  "add
  "a
  "b
  }
}

```

- **Response (HTTP 200 OK)**:

```

{

```

```
"sta
"dat
"gs
"gs
}
}
}
```

Step 6: Plans & Offers Consolidation (`GET /api/v1/onboarding/{prospectId}/plans`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "p
  :
  :
  :
  ],
  "ap
  :
  :
  ]
}
}
```

Step 7: Plan Selection (`POST /api/v1/onboarding/{prospectId}/plan-select`)

- **Request**:

```
{
  "pac
  "cou
}
```

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "p
  "s
  "c
}
```

Step 8: Calculate Amount (`GET /api/v1/onboarding/{prospectId}/calculate-amount`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "ba
  "d:
  "t:
  "c:
  "s:
  "t:
}
```

Step 9: Payment Link Generation (`POST /api/v1/onboarding/{prospectId}/payment-link`)

- **Response (HTTP 202 Accepted)**:

```
{
  "sta
  "dat
  "c:
  "a:
  "s:
  confirmat
}
```

Step 10: Document Migration (`POST /api/v1/onboarding/{prospectId}/migrate-docs`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "c:
  "m:
  "d:
}
```

Step 11: WhatsApp Consent (`POST /api/v1/onboarding/{prospectId}/consent/trigger`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "c:
  "m:
}
```

Step 12: CAF Generation (`POST /api/v1/onboarding/{prospectId}/caf`)

- **Response (HTTP 201 Created)**:

```
{
  "sta
  "dat
  "https://
  "g
  }
}
```

Step 13: EKYC Start (`POST /api/v1/onboarding/{prospectId}/ekyc/start`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "t:
  "el
  "me
  }
}
```

Status Tracking API (`GET /api/v1/onboarding/{prospectId}/status`)

- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "p
  "c
  "c
  "s
  "l
  }
}
```

Document Fetching API (`GET /api/v1/onboarding/documents`)

- **Query Parameters (Provide at least one)**:
- `prospectId` (optional)
- `customerId` (optional)
- `rmn` (optional)
- **Response (HTTP 200 OK)**:

```
{
  "sta
  "dat
  "p
  "c
  "d
  }
```

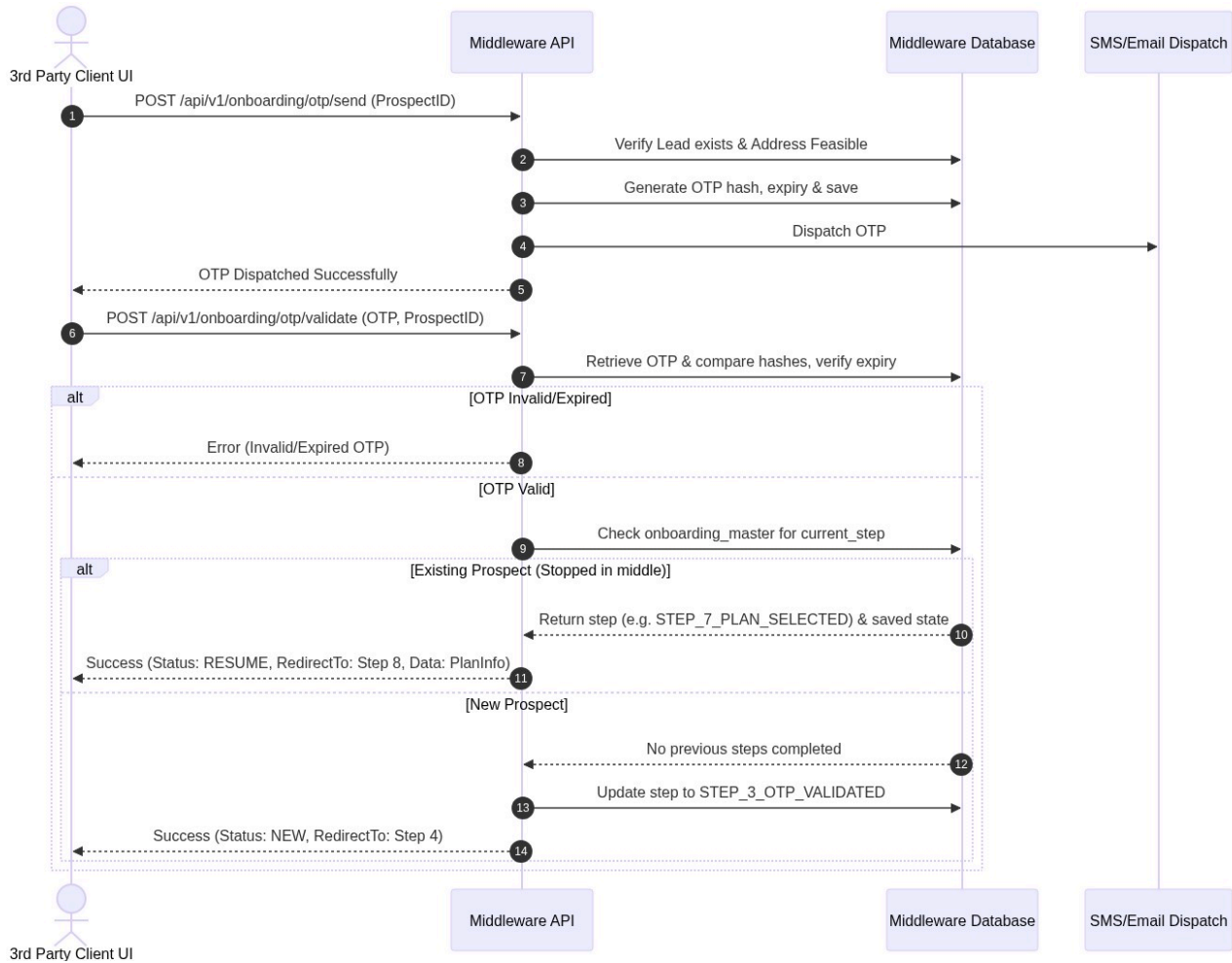
```
"https://"
```

```
"https://
"
"https://
"
"https://
"
"https://
n=..."
}
"i
"t
}
}
```

3. Sequence & Flow Orchestration Diagrams

Step 3: OTP Verification & State Recovery Flow

--- Flow Diagram 1 ---

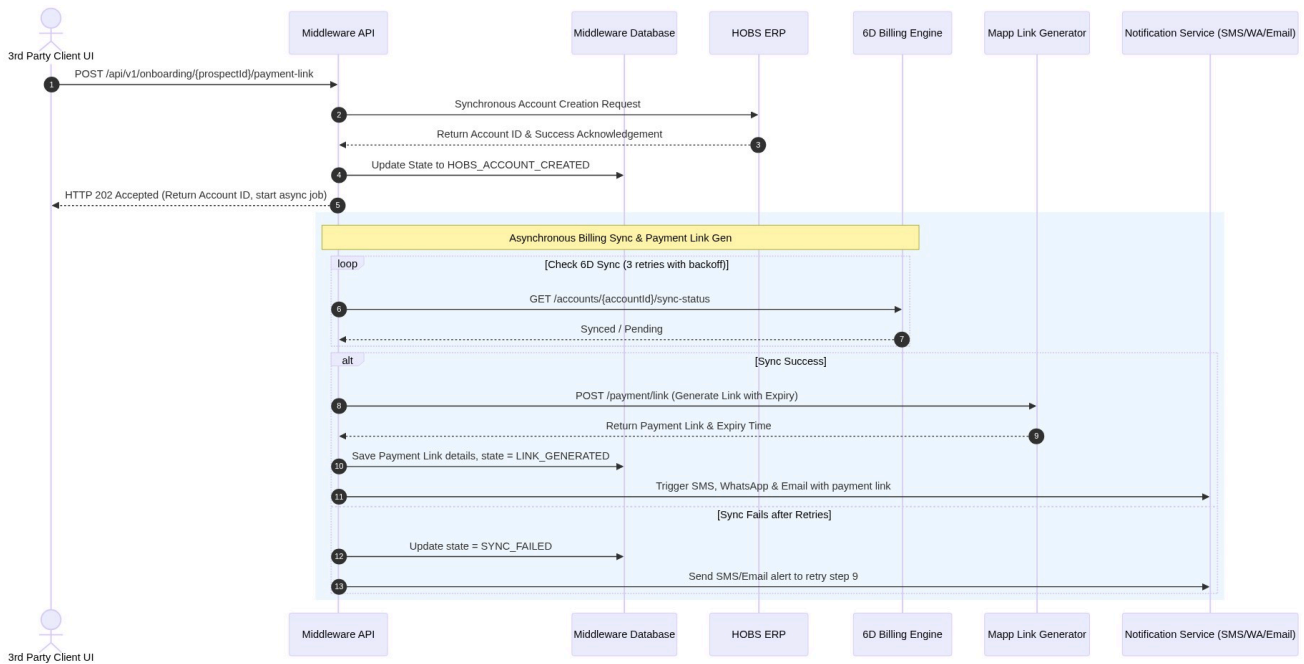


--- Onboarding Flow Table 1 ---

No.	Sender Component	Receiver Component	Interaction / Message Description	Expected Response / Outcome
1	3rd Party Client UI	Middleware API	POST /otp/send (Prospect ID)	Initiates verification check.
2	Middleware API	PostgreSQL DB	Verify lead exists & is feasible	Confirms active prospect.
3	Middleware API	PostgreSQL DB	Generate & save hashed OTP	Performs verification record.
4	Middleware API	SMS Service	Dispatch OTP request	Triggers SMS delivery.
5	SMS Service	3rd Party Client UI	Deliver 6-digit code via SMS	User receives code.
6	3rd Party Client UI	Middleware API	POST /otp/validate (OTP, ProspectID)	Validates code.
7	Middleware API	PostgreSQL DB	Retrieve OTP hash & check validity	Checks hash & expiry.
8	Middleware API	PostgreSQL DB	Check master table current_step	Determines progress stage.
9	Middleware API	3rd Party Client UI	Return NEW (Redirect to Step 4)	Frontend loads target stage.

Step 9: Asynchronous HOBS / 6D Account Creation & Payment Link Generation

--- Flow Diagram 2 ---



--- Onboarding Flow Table 2 ---

No.	Trigger / Component	Target / Integration	Action Description	Execution Type	Exception Handling
1	3rd Party UI	Middleware API	POST /payment-link	Synchronous	Standard validation error return.
2	Middleware API	HOBS ERP	Create Customer & Billing Synchron	Synchronous	Halt & fail request if HOBS is offl..
3	Middleware API	PostgreSQL DB	Update master table to HOBS_ACCOUNT_CREATED	Synchronous	Saves database state.
4	Middleware API	3rd Party UI	Return HTTP 202 Accepted (Return Account ID, start async job)	Synchronous	Provides immediate response.
5	Middleware API	6D Billing Engine	Poll billing sync status (3 retries with backoff)	Asynchronous	Log sync fail and trigger email ale..
6	Middleware API	Mapp API	POST /payment/link (Generate Link with Expiry)	Asynchronous	Lock progress and log failure.
7	Middleware API	PostgreSQL DB	Save payment link, amount, expiry	Synchronous	Updates payment record.
8	Middleware API	Notification Service	Trigger SMS, WA, Email with payment link	Synchronous	Dispatches link to customer.

4. Implementation Details for Core Controls

A. Gatekeeping Interceptor (Flow Controller)

To enforce that a customer does not bypass any onboarding step, a custom AOP aspect is registered in the Spring Boot application:

```

@Aspect
@Component
public class OnboardingFlowAspect {

    @Autowired
    private OnboardingMasterRepository repository;

    @Before("@annotation(requireStep)")
    public void verifyStepGate(JoinPoint joinPoint, RequireStep requireStep) {
        Object[] args = joinPoint.getArgs();
    }
  
```

```

UUID prospectId = extractProspectId(args);

OnboardingMaster master = repository.findById(prospectId)
    .orElseThrow(() -> new ProspectNotFoundException("Prospect ID invalid"));

StepEnum currentStep = StepEnum.valueOf(master.getCurrentStep());
StepEnum requiredPrerequisite = requireStep.value();

if (currentStep.ordinal() < requiredPrerequisite.ordinal()) {
    throw new FlowViolationException(
        "Flow violation: Required prerequisite is " + requiredPrerequisite.name()
        + ". Current stage is " + currentStep.name(),
        currentStep
    );
}
}
}

```

B. Column Encryption Converter (DPDP Compliance)

A Spring JPA Converter utilizes AES-256-GCM symmetric encryption for securing sensitive table attributes:

```

@Converter
public class PiiEncryptionConverter implements AttributeConverter<String, String> {

    @Autowired
    private EncryptionService encryptionService; // Resolved from Key Vault secret

    @Override
    public String convertToDatabaseColumn(String attribute) {
        return (attribute == null) ? null : encryptionService.encrypt(attribute);
    }

    @Override
    public String convertToEntityAttribute(String dbData) {
        return (dbData == null) ? null : encryptionService.decrypt(dbData);
    }
}

```

Applied to entity fields: Name, RMN, Email, Address, and Document numbers.